Neumann
János
Egyetem

Programozás II.

Minta ZH 2. – Labor

A feladat egy könyvtári dokumentumok adatainak feldolgozása és kezelése egy program segítségével. A dokumentumok.txt fájl tartalmazza a dokumentumok adatait, minden mező pontosvesszővel van elválasztva, és minden adatsor új sorba kerül. A fájl legfeljebb 100 adatsort tartalmaz!

Készítsen egy **Dokumentum** absztrakt osztályt, amely a példányok tárolására fog szolgálni. Az osztály minden adattagja legyen bárhonnan hozzáférhető. Az adattagok nevei a következők legyenek:

- ID – egész,
- Cim – karakterlánc,
- Kategoria – karakterlánc,
- Ar – egész,
- Darab – egész,
- KolcsonzesiIdo – egész.

Ezen felül legyen egy paraméteres konstruktora, amely az adattagokat veszi át, és átadja a megfelelő változóknak! Készítsen még egy **Osszertek** metódust, amely visszaadja a dokumentum árát. A kiszámításhoz alkalmazza az **ár * darab** képletet.

Készítsen még egy **KezelesiTerheles** absztrakt metódust, amely visszaadja a könyvtár napi terhelését. A kiszámításhoz az osztály leszármazottai eltérő szabály szerint dolgozzanak!

Írja felül a **ToString** metódust, és az alábbi formátumban adja vissza a példány adatait. Ha nem tudja a ToStringgel megoldani, akkor a **Listaz** metódusban készítse el a kódrészletet a **Konyvtar** osztályban. (Pontlevonással jár)

Minta: *[Cím] - [Kategória] ([ID]), Ára: [Ár] Ft, kölcsönzési idő: [idő] nap, darab: [darab] db*

Készítsen egy **Konyv** osztályt, amely a fenti absztrakt osztályból származik. A könyv esetében a **KezelesiTerheles** metódus az alábbi módon számoljon: *kölcsönzési idő * darab.* Készítsen egy **Folyoirat** osztályt, amely szintén a fenti absztrakt osztályból származik. A folyóirat esetében a **KezelesiTerheles** metódus az alábbi módon számoljon: *kölcsönzési idő * darab * 2*.

A két leszármazott osztály valósítsa meg az **IComparable** interfészt, és az összehasonlítás alapja az **ár** legyen! Így a dokumentumok rendezhetők legyenek ár szerint növekvő sorrendben.

Ezután hozzon létre egy **Konyvtar** nevű osztályt, amely a fentebb létrehozott osztályoknak megfelelő dokumentumokat egy **Listában** tárolja, amely bárhonnan elérhető legyen.

A nyilvános metódusok a következők legyenek:

- **Beolvas()**: a főprogramtól átvett fájl elérés alapján olvassa be a fájlt a tanult módon! Minden sorból készítsen egy objektumot. Ha a fájl nem található, kezelje az alábbi üzenettel: **„Hiba, a megadott fájl nem található!”**
  Minden más hibát elegendő általánosan kezelnie! Az adatokat a listában tárolja!

- **Listaz()**: táblázatos formában írja ki a tárolt adatokat a mintának megfelelően (ToString vagy itt megírt kód).
- **AtlagAr()**: visszaadja a tárolt dokumentumok átlagos árát.
- **Legdragabb()**: megkeresi azt a dokumentumot, amelynek a legnagyobb az ára.
- **LegkisebbTerheles()**: megkeresi azt a dokumentumot, amelynek a legkisebb a kölcsönzési terhelése.
- **KeresCimAlapjan()**: adja vissza azokat a dokumentum példányokat, amelyek címében szerepel a megadott részlet.
- **KategoriaSzerint()**: kilistázza az adott kategóriához tartozó dokumentumokat.
- **UjDokumentumHozzaadasa()**: bővíti a listát (új elem hozzáadása). Paraméterül egy Dokumentum objektumot vesz át.
- **RendezArSzerint()**: rendezi a listát ár szerint növekvő sorrendbe az IComparable megvalósítása alapján.
- **Mentes()**: kiírja a teljes adatbázist egy új fájlba, kivételkezeléssel.

Ne használjon a feladat megoldásához **File osztályt és tömböt**, valamint semmilyen olyan egyéb megoldási technikát, amelyet órai kereteken belül nem tanultunk! Ahol a függvény visszatérési értékkel rendelkezik, ott ne történjen kiíratás, csak a főfüggvényben! Csak a futó, és fordítási hiba nélküli programkódok értékelhetőek!

Egészítse ki a Program.cs-t az alábbiak szerint:

Kérje be a fájl elérését a felhasználótól, példányosítsa a **Konyvtar** osztályt, és olvassa be az adatokat!

Hívja meg az összes metódust.

[Készítsen végtelen ciklusban egy menüt a mintának megfelelően. Gondoskodjon róla, hogy gombnyomás után törlődjön a konzol tartalma, és jelenjen meg újra a menü!] → PP